IFB105 Database Management: Exam Cheat Sheet

Minh Kha (Marvin) Truong, SSP

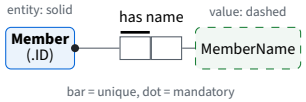
1 ORM and the CSDP

ORM = **Object Role Modelling** (not Object Relational Mapping).

CSDP steps 1 to 5

1. Turn examples into **elementary facts**.
2. Draw **fact types** and do a **population check**.
3. **Combine** entity types; mark **arithmetic** derivations *****.
4. Add **uniqueness** bars; check **arity**.
5. Add **mandatory** dots; mark **logical** derivations *****.

Notation



- **Entity type**: needs a reference mode, e.g. `(.ID)`, `(.Code)`.
- **Value type**: identifies itself (a string or number). Never has a reference mode.
- **Unit-based** modes `(AUD:)`, `(L:)`, `(cm:)` make an **entity** type.
- **Arity** = number of roles: unary 1, binary 2, ternary 3.

Step 1: elementary facts

Pattern: *entity type + reference mode + has/is + value type + value*.
The Member with ID 'M01' has the MemberName 'Ava Tran'.

- Can't be split without losing information.
- States something **true** (no "is not").
- Has **no conditions** (no "if", "so").
- A **tick-box** on a form is a **unary** fact, not a Yes/No value.
- Count **object types** only; not values or reference modes.

Step 2: population check

Write 3+ sample rows under the role boxes.

- Repeats in a column ⇒ role is **not** unique.
- Object missing from a column ⇒ role is **optional**.
- Can't fill a row ⇒ fact type is wrong; go back to Step 1.

Nest a fact type (m:n bar) when facts are recorded about the pair itself, e.g. Booking *was made on* Date.

Step 3: combine and derive

Combine types if the same entity can be both, they are comparable, or you'd list them together. Use **role names** `[retailPrice]` to keep predicates apart. Derivable facts get ***** and a rule, and are **not stored**.

1 ORM: constraints (steps 4 and 5)



- Bar = values in that role (or combination) **never repeat**.
- n:1 bar goes over the role that **never repeats**.
- Every binary fact type needs **at least one** bar.
- **n-1 rule**: each bar spans at least $n-1$ roles. On a ternary, every bar covers 2+ roles (a dotted gap skipping a role still counts). A bar on 1 role of a ternary ⇒ split or nest.
- **Mandatory dot**: every instance **must** play the role. A gap in data proves optional; no gap does **not** prove mandatory. Check the wording ("must", "required").

> **Constraint checks (Q19).** A dot means *every* object known to the database plays that role, including ones that only appear in another fact type. Adding a fact that brings in a new student who isn't enrolled breaks the dot.

2 Relational Mapping (RMapping)

RMap syntax

`Member {memberID, name, [phone], email}`

- `[brackets]` = optional (NULL allowed).
- Underline = unique. **Double underline** = PK when another column is also underlined.
- Dashed arrow = foreign key, child column → parent key.

The four rules

1. Bar spans 2+ roles ⇒ own table named after the predicate; PK = spanned roles.
2. Single-role bars on one object ⇒ group them into one table for that object; PK = reference mode.
3. Nested fact type ⇒ table named after the nest; PK = its spanning bar, plus facts attached to it.
4. One-to-one ⇒ goes in exactly **one** table (see below).

2 RMapping: one-to-one and procedure

Rule 4 decision tree

- **4.1** Only one object has another functional role ⇒ put it in that object's table.
- **4.2** Both have other roles, only one side mandatory ⇒ the **mandatory** side.
- **4.3** Neither has another functional role ⇒ separate table.
- **4.4** Both have roles, both or neither mandatory ⇒ your choice; **fewest NULLs**.

Procedure

1. Cross out derived (*) fact types.
2. Rule 1 for non-nested spanning bars.
3. Rule 3 for nested fact types.
4. Rule 2 for each object type.
5. Rule 4 for 1:1; underline the non-key side.
6. No dot ⇒ `[brackets]`.
7. Drop tables holding only a key (Room, Date, Language).
8. Draw every foreign key.

FitHub, mapped

```
Member {memberID, memberName, [phoneNo], email}
Booking {memberID, classCode, bookedOn}
Class {classCode, className, trainerID, roomNr}
Trainer {trainerID, trainerName, licenceNr}
Speaks {trainerID, languageName}
```

> Rule 4 depends on the object's *other* roles. User *has* email with no other functional role ⇒ 4.3 (own table). Member also has a name ⇒ 4.1 (email goes in Member).

5 CARE principles (Q16)

- **C** **Collective benefit**: does use of the data benefit the community it came from?
- **A** **Authority to control**: do communities have a say in access and use?
- **R** **Responsibility**: are relationships kept up over time?
- **E** **Ethics**: are rights and wellbeing central; were they consulted on benefits, harms, future use?

For each: name the principle, say met or not met, point to the fact in the scenario.

Exam traps

- Answer the step asked: Q18 is Steps 1 and 2 only, so no bars, no dots, keep UnitPrice.
- Always write the reference mode: "The **Member with ID** 'M01'", not "M01".
- `IS NULL`, never `= NULL`.
- `UPDATE`/`DELETE` without `WHERE` hits every row.
- `ALTER` changes structure; `UPDATE` changes data.
- `IN` for multi-row subqueries; `=` only for exactly one row.
- Strings in single quotes; numbers without.
- Match table name case exactly (Linux is case-sensitive).

3 Normalisation

Removes redundancy bottom-up from a messy table. Ends at the same tables as ORM + RMapping.

Anomalies

Insertion can't add data without unrelated data
Deletion deleting one thing loses facts about another
Update a repeated fact changed in some rows only

Keys

- **Super key**: any unique set of columns.
- **Candidate key**: minimal super key (remove any column ⇒ duplicates).
- **Primary key**: the chosen candidate key.
- **Foreign key**: child column referencing a parent key.
- Every candidate key is a super key, not the other way round.

Dependencies

- $A \to B$: each A has exactly one B. One-way only.
- **Partial**: depends on *part* of a composite PK.
- **Transitive**: non-key → non-key (key → $A \to B$).
- **Full**: needs the whole key.

Normal forms

1NF Has a PK; one type per column; values **atomic**, no repeating groups. Fix: one row per repeated item.

2NF 1NF + **no partial** dependencies. Fix: move them out with the part of the key they depend on. Only matters with a **composite** PK.

3NF 2NF + **no transitive** dependencies. Fix: new table keyed on the determinant; keep it as an FK.

Highest normal form: check in order



Test dependencies **on the data**: find two rows with the same X; different Y means $X \not\to Y$. Ignore column order, and never use names as keys (two different patients are both "Mary Smith").

A column with gaps in the data is optional `[col]`. An m:n pair (book with two authors, author with two books) needs its own table.

4 SQL: DDL and DML

RMap to CREATE TABLE

underlined key	`PRIMARY KEY` / `PRIMARY KEY (a, b)`
no brackets	`NOT NULL`
`[brackets]`	leave `NOT NULL` off
underlined, not PK	`UNIQUE`
dashed arrow	`FOREIGN KEY (c) REFERENCES T(c)`
fixed list	`ENUM('A', 'B')`
generated number	`INT AUTO_INCREMENT`

```
CREATE TABLE Booking (
  memberID  VARCHAR(5) NOT NULL,
  classCode VARCHAR(6) NOT NULL,
  bookedOn  DATE       NOT NULL,
  PRIMARY KEY (memberID, classCode),
  FOREIGN KEY (memberID)  REFERENCES Member(memberID),
  FOREIGN KEY (classCode) REFERENCES Class(classCode));
ALTER TABLE Member ADD COLUMN tier ENUM('Std','Pro');
```

Create parents first; drop children first.

DML

```
INSERT INTO Member VALUES ('M04','Dan Ho',NULL,'d@m.com');
INSERT INTO Member (memberID, memberName, email)
  VALUES ('M05','Eve Lam','e@m.com');
UPDATE Member SET phoneNo = '0400' WHERE memberID = 'M02';
DELETE FROM Booking WHERE memberID = 'M03';
```

Will this INSERT run? Check in order

PK duplicate? → **FK** parent exists? → **UNIQUE** taken? → **NOT NULL** missing? → **ENUM** allowed value? Deleting a parent with child rows also fails.

4 SQL: queries

Execution order

FROM/JOIN → **WHERE** → **GROUP BY** → **HAVING** → **SELECT** → **ORDER BY**

Operators

`IN (...)`, `BETWEEN a AND b` (inclusive), `LIKE 'C%'` (`%` any, `_` one), `IS NULL`, `DISTINCT`, `ORDER BY c DESC`.

Aggregates

```
SELECT customerID, SUM(totalPrice) AS spent
FROM Orders
GROUP BY customerID
HAVING SUM(totalPrice) > 100;
```

- WHERE filters rows; HAVING filters groups.
- `COUNT(*)` counts rows; `COUNT(col)` skips NULLs.
- Non-aggregated SELECT columns go in GROUP BY.

Joins

```
SELECT m.memberName FROM Member m
LEFT JOIN Booking b ON m.memberID = b.memberID
WHERE b.classCode IS NULL;   -- never booked
```

INNER keeps matches only. LEFT keeps every left row; unmatched right columns are NULL.

Subqueries and set operators

```
SELECT className FROM Class WHERE classCode NOT IN
  (SELECT classCode FROM Booking WHERE memberID = 'M01');
SELECT memberID FROM Member
EXCEPT SELECT memberID FROM Booking;
```

Read nested `IN` from the inside out. `UNION` removes duplicates, `UNION ALL` keeps them, `EXCEPT` subtracts. Same number of columns on both sides.

4 SQL: TCL and DCL

```
START TRANSACTION;
UPDATE Member SET phoneNo = '0499' WHERE memberID = 'M04';
COMMIT;   -- or ROLLBACK;
CREATE ROLE 'frontdesk';
GRANT SELECT ON fithub.* TO 'frontdesk';
GRANT INSERT, UPDATE ON fithub.Booking TO 'frontdesk';
GRANT 'frontdesk' TO 'sam'@'localhost';
SET DEFAULT ROLE ALL TO 'sam'@'localhost';
REVOKE UPDATE ON fithub.Booking FROM 'frontdesk';
```

Two roles = union of privileges. Add/edit but not remove: grant `SELECT`, `INSERT`, `UPDATE`, no `DELETE`.

CREATE, ALTER, DROP commit automatically. A later ROLLBACK can't undo anything before them.